

HandControlPC

Contrôle gestuel du PC par webcam

Mohamed Amine Sobhi

Cnam Occitanie — NFP101 Programmation Orientée Objet
Professeur : Adrien Escourrou

Mai 2026

Plan de la présentation

- ➊ Introduction, contexte et objectif
- ➋ Les étapes du projet
- ➌ Démonstration et résultat
- ➍ Architecture technique
- ➎ Validation et tests
- ➏ Discussion, limites et IA
- ➐ Bilan et références

Contexte : pourquoi HandControlPC ?

Le constat

- Clavier et souris imposent un contact physique permanent.
- Certains contextes y sont mal adaptés : présentation, accessibilité, mains occupées.

L'idée

- Utiliser la **webcam** déjà présente sur chaque PC.
- Traduire des **gestes de la main** en actions système.
- Aucun matériel supplémentaire.

Public cible

- Utilisateurs en présentation
- Accessibilité (handicap moteur)
- Curieux de la vision par ordinateur

Objectif du projet

Fonctionnalité centrale

Capter les gestes de la main en temps réel et les convertir en actions concrètes : **volume**, **capture d'écran**, **zoom**, et **reconnaissance de signes**.

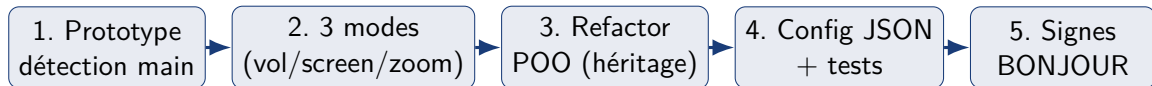
Objectifs techniques

- Application *entièrement finie* (pas un prototype)
- Architecture orientée objet propre
- Configuration externe
- Tests automatisés

Contraintes

- Temps réel (> 30 FPS)
- Robustesse aux faux positifs
- Code lisible et extensible

Les diverses étapes du projet



- Démarche **incrémentale** : chaque étape ajoute une fonctionnalité démontrable.
- Suivi par **commits Git réguliers** (preuve d'avancement).
- Refactorisation POO *après* avoir validé le fonctionnel.

Démonstration — les 4 modes

Modes de contrôle

- ① **Volume** : pincement pouce-index
- ② **Screenshot** : 3 doigts maintenus
- ③ **Zoom** : écartement de 2 mains
- ④ **Signes** : épeler BONJOUR

Résultats observables

- Volume système modifié en direct
- Fichier PNG horodaté + son
- Flux vidéo zoomé en temps réel
- Synthèse vocale « Bonjour »

[Démonstration en direct — ~3 minutes]

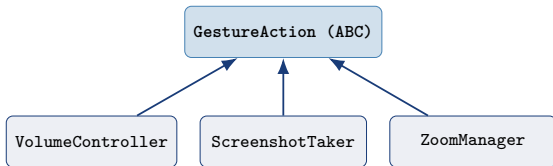
Lancer `python main.py` puis `python test_sign_recognition.py`

Le geste « BONJOUR » en pratique

B	O	N	J	O	U	R
4 doigts	3 du milieu	pouce+2	shaka	3 du milieu	V	index

- Chaque lettre valide après **20 frames stables** ($\sim 0,7$ s).
- Affichage progressif : **vert** = validé, **jaune** = attendu, gris = à venir.
- Une erreur de séquence remet le compteur à zéro.
- Séquence complète $\rightarrow$ synthèse vocale + message de succès.

Architecture : héritage et polymorphisme



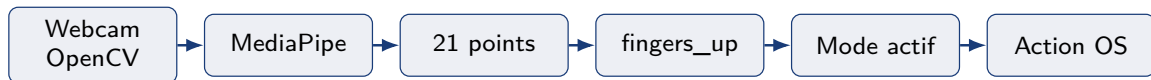
Choix clés

- `process()` **abstraite** : chaque mode sa logique.
- `render_hud()` **héritée** : affichage commun.
- **Open/Closed** : ajouter un mode = une sous-classe.

Polymorphisme

`main.py` manipule les actions via l'interface commune `GestureAction`.

Pipeline temps réel et stack technique



Bibliothèques

- **MediaPipe** : 21 landmarks/main
- **OpenCV** : vidéo + HUD
- **NumPy** : interp / clip
- **MSS** : capture d'écran

Configuration externe

config.json centralise tous les paramètres : résolution, seuils, dossiers. Chargement avec deepcopy + fallback.

29 tests unitaires (pytest)

- **Sans webcam** : dépendances mockées.
- Signes, séquence BONJOUR, volume, zoom, config.
- `python -m pytest tests/ -v`

Résultat

29 passed en $\sim 1,5$ s

Robustesse « en direct »

- **Cooldown** entre actions
- **Historique glissant** (anti faux positif)
- Seuil de confiance $\geq 0,6$
- try/except sur chaque frame

Discussion — limites et axes d'amélioration

Limites

- macOS uniquement (osascript, afplay)
- Sensible à la luminosité
- Signes *statiques* seulement
- Mono-utilisateur

Pistes

- Contrôle souris par geste
- Mode apprentissage de gestes
- Portage Windows/Linux
- Logs persistants

IA déclarées

Code assisté encadré par # CODE IA (Claude) dans les fichiers sources concernés.

Où l'IA est marquée

- **Claude** — `volume_control.py`
(classe + `_get/_set_volume`)
- **Claude** — `zoom_control.py`
(méthode `adjust()`)
- **GitHub Copilot** — `main.py`
(refactorisation légère)

Ce que j'ai maîtrisé

- Logique de détection (`fingers_up`)
- Choix itératif des patterns
- Intégration MediaPipe/OpenCV
- Architecture POO et tests

Je peux expliquer chaque passage assisté devant le jury.

Ce que ce projet m'a apporté

Compétences acquises

- Conception **POO** concrète (ABC, héritage, polymorphisme)
- Vision par ordinateur temps réel
- Tests automatisés et mocking
- Gestion de configuration

Applicable en entreprise ?

Oui :

- Architecture extensible (OCP)
- Tests → intégration continue
- Config externe → déploiement
- Traçabilité (Git, doc, IA déclarée)

Ce que j'aurais changé dans l'énoncé

- + Sujet libre : **très motivant**, permet de choisir un domaine qui passionne.
- + Exigences claires (POO, tests, doc) : bon cadre pédagogique.
- ~ Préciser un **ordre de grandeur** attendu (nb de classes, lignes) pour cadrer la portée.
- ~ Proposer une **liste d'exemples** de sujets pour ceux qui hésitent.
- ~ Donner un **barème détaillé des bonus** pour mieux prioriser.

Bibliographie et références

Bibliothèques et documentation

- **MediaPipe** (Google) — détection de mains
- **OpenCV** — vision par ordinateur
- **Python ABC** — classes abstraites
- **pytest** — tests unitaires
- **MSS** — capture d'écran

Intelligence artificielle (déclarée)

- **Claude Code** (Anthropic) — `volume_control.py`, `zoom_control.py`
- **GitHub Copilot** — refactorisation légère de `main.py`

Merci de votre attention

Questions & réponses

HandControlPC — Mohamed Amine Sobhi

Cnam NFP101 — Mai 2026